Descripción de componentes: Card y Tooltip

Diego Alejandro Botina

20 de octubre de 2026

Índice

0.1 1. Introducción 2
0.2 2. Componente Card 2
0.2.1 2.1 Descripción general 2
0.2.2 2.2 Estructura HTML 2
0.2.3 2.3 Estilización con CSS 3
0.2.4 2.4 Funcionalidad JavaScript 3
0.2.5 2.5 Pruebas 5
0.3 3. Componente Tooltip 5
0.3.1 3.1 Descripción general 5
0.3.2 3.2 Estructura HTML 5
0.3.3 3.3 Estilización con CSS 6
0.3.4 3.4 Funcionalidad JavaScript 6
0.3.5 3.5 Pruebas 8
0.4 4. Sistema de tokens de diseño 8
0.5 5. Pruebas y control de calidad 8
0.6 6. Conclusión 9



Figura 1: Logo Jala University

Universidad: Jala University
Curso: Desarrollo de Software IV
Docente: Juan Esteves
Estudiante: Diego Alejandro Botina
Fecha de entrega: 20 de octubre de 2026

Repositorio: https://gitlab.com/jala-university1/cohort-5/ES.CSSD-245.GA.T2.26.M2/SD/group-b/individual-laboratories/alejandro-botina/week-03/jala-u-ds4-web-components

0.1 1. Introducción

El presente documento describe los dos componentes web desarrollados para el laboratorio de la semana 3: `Card` y `Tooltip`. Ambos forman parte de una librería reutilizable construida exclusivamente con HTML5, CSS3 y JavaScript puro, sin dependencias en tiempo de ejecución. La librería se publica bajo el nombre `@jalau-labs/jala-u-ds4-web-components` en el registro de npm.

Cada componente se diseñó siguiendo los siguientes principios:

- **Separación de responsabilidades.** El HTML, el CSS y el JavaScript de cada componente viven en archivos independientes dentro de su propio directorio.
- **Reutilización mediante tokens de diseño.** Ningún componente define colores, tamaños o espaciados de forma literal: todos los valores provienen de custom properties CSS centralizadas en `src/styles/tokens.css`.
- **Manipulación segura del DOM.** No se utiliza `innerHTML` en ningún punto. Todo el árbol de nodos se construye con `document.createElement`, eliminando vectores de inyección de HTML.
- **Accesibilidad desde el diseño.** Los componentes incorporan atributos ARIA, gestión de foco y soporte de teclado.
- **Documentación exhaustiva.** Cada método público de JavaScript está documentado con JSDoc.
- **Cobertura de pruebas.** Cada componente cuenta con su propia batería de pruebas unitarias ejecutadas con Vitest sobre jsdom.

0.2 2. Componente Card

0.2.1 2.1 Descripción general

El componente `Card` agrupa un título, un subtítulo opcional, un cuerpo de texto opcional y una lista opcional de botones de acción dentro de un contenedor visualmente delimitado. Se utiliza para presentar información relacionada que debe leerse como una unidad.

El componente soporta tres variantes visuales:

Variante	Descripción
`default`	Borde sutil y fondo de superficie por defecto.
`outlined`	Borde más grueso y fondo transparente.
`elevated`	Sin borde, con sombra proyectada.

0.2.2 2.2 Estructura HTML

El componente genera un elemento `<article>` raíz con la clase `jc-card` y una clase modificadora que identifica la variante. La estructura interna sigue el patrón BEM (*Block Element Modifier*):

```
<article class="jc-card jc-card--elevated" role="article">
  <header class="jc-card__header">
```

```
    <h2 class="jc-card__title">Título de la tarjeta</h2>
    <p class="jc-card__subtitle">Subtítulo opcional</p>
  </header>
  <div class="jc-card__body">Contenido del cuerpo</div>
  <footer class="jc-card__footer">
    <button type="button" class="jc-card__action jc-card__action--
secondary">Cancelar</button>
    <button type="button" class="jc-card__action jc-card__action--
primary">Confirmar</button>
  </footer>
</article>
```

Se emplean elementos semánticos (`article`, `header`, `footer`) en lugar de `<div>` genéricos siempre que la semántica lo permite. El título se renderiza con `<h2>` para integrarse coherentemente en la jerarquía de encabezados de la página que lo consume.

0.2.3 2.3 Estilización con CSS

Los estilos del componente se definen en `src/components/card/card.css`. Ninguna regla contiene valores literales de color, tamaño o espaciado: todos se obtienen de custom properties definidas en `src/styles/tokens.css`. Esto permite que un consumidor cambie la apariencia completa del componente redefiniendo únicamente los tokens.

Ejemplo de regla que ilustra el patrón:

```
.jc-card {
  display: flex;
  flex-direction: column;
  gap: var(--jc-space-4);
  padding: var(--jc-card-padding);
  border-radius: var(--jc-card-radius);
  background-color: var(--jc-color-bg-surface);
  color: var(--jc-color-text-primary);
}
```

Las variantes se implementan mediante clases modificadoras independientes (`.jc-card--default`, `.jc-card--outlined`, `.jc-card--elevated`), evitando anidamiento excesivo y facilitando la extensión.

0.2.4 2.4 Funcionalidad JavaScript

El componente se implementa como una clase ES6 exportada desde `src/components/card/card.js`. La clase encapsula su estado mediante campos privados (`#options`, `#element`, `#actionListeners`) y expone únicamente los métodos documentados.

0.2.4.1 API pública

`constructor(options)`

Crea una instancia de `Card`. Valida que las opciones sean coherentes antes de almacenarlas.

Parámetro	Tipo	Obligatorio	Descripción
`options.title`	`string`	Sí	Título de la tarjeta. No puede estar vacío.
`options.subtitle`	`string`	No	Subtítulo opcional.
`options.body`	`string`	No	Texto del cuerpo.
`options.actions`	`CardAction[]`	No	Lista de acciones (botones).
`options.variant`	`'default' | 'outlined' | 'elevated'`	No	Variante visual. Por defecto `'default'`.

Cada `CardAction` tiene la forma:

```
{
  label: 'Confirmar',        // texto visible
  onClick: (event) => {...}, // manejador del clic
  variant: 'primary'         // 'primary' | 'secondary' (por defecto 'secondary')
}
```

Errores que lanza el constructor:

- `TypeError` si `options` no es un objeto, si `title` no es una cadena no vacía, o si `actions` no es un arreglo.
- `RangeError` si `variant` no es uno de los valores soportados.

`render()`

Construye el árbol de nodos del componente y lo devuelve como un `HTMLElement`. La llamada es idempotente: invocarla varias veces devuelve siempre la misma instancia del elemento. El elemento no se inserta automáticamente en el DOM; el consumidor decide dónde montarlo.

`destroy()`

Elimina todos los listeners registrados por la instancia y desprende el elemento del árbol del DOM si estaba montado. Es seguro llamarlo varias veces. Después de `destroy()`, el getter `element` devuelve `null`.

Getter `element`

Devuelve el `HTMLElement` renderizado o `null` si el componente aún no se ha renderizado o ya fue destruido.

0.2.4.2 Gestión de eventos

Cada botón de acción registra un único listener del tipo `click`. La referencia al listener se guarda en un `Map` interno (`#actionListeners`) indexado por el propio botón, lo que permite retirar todos los listeners durante `destroy()` sin posibilidad de dejar referencias huérfanas.

0.2.4.3 Ejemplo de uso

```
import { Card } from '@jalau-labs/jala-u-ds4-web-components';

const card = new Card({
  title: 'Resumen semanal',
  subtitle: 'Semana del 14 al 20 de octubre',
  body: 'Todas las tareas se completaron correctamente.',
  variant: 'elevated',
  actions: [
    { label: 'Descartar', onClick: () => card.destroy() },
    { label: 'Confirmar', onClick: () => console.log('confirmado'), variant:
'primary' },
  ],
});

document.querySelector('#contenedor').appendChild(card.render());
```

0.2.5 2.5 Pruebas

El archivo `src/components/card/card.test.js` contiene 21 pruebas organizadas en cuatro bloques: `constructor`, `render`, `actions` y `destroy`. Se cubren casos de validación de entrada, generación correcta del árbol de nodos, asignación de manejadores de eventos, retiro de listeners durante la destrucción, idempotencia de `render` y `destroy`, y escape de caracteres HTML en el contenido textual. La cobertura final del archivo `card.js` es del 100% en sentencias, ramas, funciones y líneas.

0.3 3. Componente Tooltip

0.3.1 3.1 Descripción general

El componente `Tooltip` asocia un texto breve a un elemento objetivo y lo muestra al pasar el cursor por encima o al enfocar el elemento con el teclado. Se utiliza para aportar contexto adicional sin recargar visualmente la interfaz.

El tooltip soporta cuatro posiciones de anclaje: `top`, `right`, `bottom` y `left`. Por defecto utiliza `top`.

0.3.2 3.2 Estructura HTML

A diferencia del `Card`, el `Tooltip` no se monta como hijo del elemento objetivo. Se renderiza en un elemento *portal* independiente que se añade directamente a `document.body`. Esta decisión evita problemas de recorte cuando el elemento objetivo se encuentra dentro de un contenedor con `overflow: hidden`, y permite posicionarlo con coordenadas absolutas relativas al viewport.

Estructura generada:

```
<div
  id="jc-tooltip-1"
  class="jc-tooltip jc-tooltip--top jc-tooltip--visible"
  role="tooltip"
  aria-hidden="false"
>
```

```
  <span class="jc-tooltip__arrow" aria-hidden="true"></span>
  <span class="jc-tooltip__content">Texto del tooltip</span>
</div>
```

Además, cuando se llama a `attach(target)`, el componente escribe el atributo `aria-describedby` sobre el elemento objetivo, enlazándolo con el identificador del tooltip. Esto permite que los lectores de pantalla anuncien automáticamente el texto del tooltip cuando el usuario enfoca el objetivo.

0.3.3 3.3 Estilización con CSS

El tooltip utiliza `position: fixed` y se posiciona mediante las propiedades `top` y `left`, calculadas en JavaScript a partir del rectángulo del elemento objetivo (`getBoundingClientRect`). El CSS se apoya íntegramente en tokens de diseño, con la misma filosofía que el `Card`.

Una particularidad es la flecha del tooltip, que se implementa como un cuadrado rotado 45 grados:

```
.jc-tooltip__arrow {
  position: absolute;
  width: var(--jc-tooltip-arrow-size);
  height: var(--jc-tooltip-arrow-size);
  background-color: var(--jc-color-text-primary);
  transform: rotate(45deg);
}
```

La visibilidad se controla con una clase modificadora `.jc-tooltip--visible` que alterna la opacidad. La transición entre ambos estados se delega a una regla `transition` sobre la propiedad `opacity`, aprovechando el token `--jc-duration-fast`.

0.3.4 3.4 Funcionalidad JavaScript

La clase `Tooltip` se exporta desde `src/components/tooltip/tooltip.js`. Al igual que `Card`, encapsula su estado en campos privados y expone una API mínima.

0.3.4.1 API pública

`constructor(options)`

Parámetro	Tipo	Obligatorio	Descripción
`options.text`	`string`	Sí	Texto del tooltip. No puede estar vacío.
`options.position`	`'top' | 'right' | 'bottom' | 'left'`	No	Posición relativa al objetivo. Por defecto `'top'`.

Errores que lanza:

- `TypeError` si `options` no es un objeto o si `text` no es una cadena no vacía.
- `RangeError` si `position` no es uno de los valores soportados.

`attach(target)`

Asocia el tooltip a un elemento objetivo. Registra los listeners `mouseenter`, `mouseleave`, `focus`, `blur`, `keydown` (para detectar `Escape`) y `scroll`/`resize` sobre `window` (para ocultar el tooltip cuando el layout cambia). Escribe el atributo `aria-describedby` sobre el objetivo. La llamada es idempotente cuando se repite con el mismo objetivo, y desasocia el anterior si se invoca con uno distinto.

`show()`

Muestra el tooltip. La primera vez que se invoca, construye el elemento y lo añade a `document.body`; en invocaciones posteriores reutiliza el mismo elemento. Calcula la posición a partir del rectángulo del objetivo y de las dimensiones actuales del propio tooltip.

`hide()`

Oculta el tooltip retirando la clase modificadora y ajustando `aria-hidden`. El elemento permanece en el DOM para hacer económica la siguiente aparición.

`destroy()`

Retira el elemento del DOM, elimina todos los listeners y limpia el atributo `aria-describedby` del objetivo. Es seguro llamarlo varias veces.

Getters `element`, `target` e `id`

Devuelven respectivamente el elemento renderizado, el objetivo asociado y el identificador único de la instancia. El getter `id` expone el valor que se escribe en `aria-describedby`, lo que permite a un consumidor referenciar el tooltip desde su propio marcado ARIA.

0.3.4.2 Posicionamiento

El cálculo de coordenadas se realiza en el método privado `#positionElement`. Para cada posición se aplica una fórmula distinta sobre el rectángulo del objetivo y las dimensiones del tooltip:

- `top`: `top = target.top - tooltip.height`, `left = target.left + target.width/2 - tooltip.width/2`
- `bottom`: `top = target.bottom`, `left` igual al caso anterior
- `left`: `top = target.top + target.height/2 - tooltip.height/2`, `left = target.left - tooltip.width`
- `right`: `top` igual al caso anterior, `left = target.right`

Los resultados se aplican como estilos en línea (`style.top` y `style.left`).

0.3.4.3 Gestión de eventos

Los listeners se guardan en un `Map` interno, igual que en `Card`. El listener de `scroll` se registra con la opción `capture: true` para detectar también el desplazamiento de contenedores anidados, no solo el de la ventana.

0.3.4.4 Ejemplo de uso

```
import { Tooltip } from '@jalau-labs/jala-u-ds4-web-components';

const tooltip = new Tooltip({
```

```
  text: 'Copia el valor al portapapeles',
  position: 'top',
});

tooltip.attach(document.querySelector('#boton-copiar'));
```

0.3.5 3.5 Pruebas

El archivo `src/components/tooltip/tooltip.test.js` contiene 26 pruebas distribuidas en cinco bloques: `constructor`, `attach`, `show / hide`, `event listeners`, `destroy` e `ids`. Se mockea `getBoundingClientRect` mediante una función auxiliar para verificar la matemática del posicionamiento en las cuatro variantes, ya que jsdom no calcula dimensiones reales. Se cubren también los ciclos completos de interacción con teclado, ratón, resize de ventana, y la generación de identificadores únicos entre instancias.

0.4 4. Sistema de tokens de diseño

La librería define sus valores visuales en dos capas superpuestas de custom properties CSS, declaradas en `src/styles/tokens.css` bajo el selector `:where(:root)`.

- **Capa primitiva.** Contiene la paleta cruda y las escalas (`--jc-color-blue-500`, `--jc-space-4`, `--jc-radius-lg`, etc.). No debe consumirse directamente desde los componentes.
- **Capa semántica.** Alias con significado (`--jc-color-text-primary`, `--jc-color-bg-surface`, `--jc-card-padding`, etc.). Los componentes usan exclusivamente esta capa.

Separar ambas capas permite dos cosas: cambiar el tema completo reasignando solo la capa semántica, y exponer los tokens al consumidor con una especificidad de cero, de modo que pueda sobrescribirlos sin recurrir a `!important`.

El archivo incluye además un bloque `@media (prefers-color-scheme: dark)` que reasigna los tokens semánticos cuando el sistema operativo del usuario está en modo oscuro.

0.5 5. Pruebas y control de calidad

Cada componente cuenta con su propia suite de pruebas unitarias. La infraestructura se apoya en:

- **Vitest** como corredor de pruebas, con soporte nativo de módulos ES.
- **jsdom** como entorno DOM para las pruebas, permitiendo manipular `document` y `window` sin necesidad de un navegador.
- **`@vitest/coverage-v8`** como motor de cobertura, aprovechando el soporte nativo de Node para reportar líneas ejecutadas.

Los archivos de prueba viven junto a los archivos de implementación (`card.test.js` junto a `card.js`), siguiendo la convención de "pruebas cercanas al código" que facilita el mantenimiento.

El pipeline de integración continua ejecuta la secuencia completa en cada push a la rama `main`, tanto en GitHub Actions como en GitLab CI. La secuencia es idéntica al comando `make ci` que se puede correr localmente:

1. `eslint .` — análisis estático.
2. `prettier --check .` — verificación de formato.

3. `vitest run --coverage` — pruebas unitarias con cobertura.
4. `vite build` — generación de los artefactos de distribución.

Estado actual: 49 pruebas pasando, cobertura combinada del 98% en sentencias.

0.6 6. Conclusión

El proyecto entrega dos componentes web reutilizables, con documentación JSDoc exhaustiva, pruebas unitarias automatizadas y empaquetado listo para publicación en npm. La arquitectura sigue una separación estricta entre estructura, presentación y comportamiento, y todas las decisiones visuales se canalizan a través de un sistema de tokens que el consumidor puede sobreescribir sin modificar la librería.

El código fuente completo, el historial de cambios y la configuración de integración continua están disponibles en el repositorio enlazado en la portada.